PL/I CHEAT SHEET

Core setup, initialization, operations & integration quick reference

PART 1: SETUP & CORE

01. OVERVIEW & INSTALL

Installation

PL/I modern, expressive programming language

```
# Check version
pl/i --version
```

Install via official installer, package manager, or version manager
Popular package managers: cargo, pip, gem, npm, brew

02. BASIC SYNTAX

Architecture

Variables, control flow, expressions

```
// Variable declaration
let x = 10;
// Conditional
if x > 5 { ... } else { ... }
// Loop
for item in collection { ... }
```

03. DATA TYPES

YAML / Config

Primitive: integers, floats, booleans, strings
Composite: arrays, maps, tuples, structs

```
// Typed declaration
let name: String = "Alice";
let numbers: Vec<i32> = vec![1, 2, 3];
```

Strong typing prevents common runtime errors

04. FUNCTIONS

Workflow

```
fn greet(name: &str) -> String {
format!("Hello, {}!", name)
}
let result = greet("World");
```

First-class functions; lambdas/closures supported

```
let double = |x| x * 2;
```

05. OOP / MODULES

CRUD API

Structs, classes, or records with methods

```
struct User { name: String, age: u32 }
impl User {
fn new(name: &str) -> Self { ... }
fn greet(&self) -> String { ... }
}
```

Interfaces / traits / protocols for polymorphism

06. COLLECTIONS

Integration

```
// Array / slice
let arr = [1, 2, 3];
// Dynamic list
let mut list = Vec::new();
list.push(42);
// Map / dictionary
let mut map = HashMap::new();
map.insert("key", "value");
```

SECURITY, MONITORING & OPERATIONS

Production hardening, observability, performance tuning & CLI reference

PART 2: OPS & SECURITY

07. ERROR HANDLING

Hardening

Result/Option types or exceptions

```
fn read_file(path: &str) -> Result<String, Er...
Ok(std::fs::read_to_string(path)?)
}
match result {
Ok(data) => process(data),
Err(e) => eprintln!("Error: {}", e),
}
```

08. CONCURRENCY / ASYNC

Observability

Threads, async/await, or actor model

```
async fn fetch_data(url: &str) -> Result<Stri...
let resp = client.get(url).await?;
resp.text().await
}
tokio::main / async_std / smol
```

Green threads or OS threads depending on runtime

09. TESTING

Optimization

Built-in or framework test support

```
#[test]
fn test_greet() {
assert_eq!(greet("Alice"), "Hello, Alice!");
}
cargo test  /  run test suite
```

Unit tests co-located with source code

10. ECOSYSTEM & PACKAGES

Unit Tests

Package registry and dependency management

```
# Add dependency
cargo add serde
# In Cargo.toml:
[dependencies]
serde = { version = "1.0", features = ["deriv...
```

Curated ecosystem with popular libraries for HTTP, JSON, DB

11. CLI & BUILD TOOLS

Commands

```
cargo build   compile project
cargo run     build and run
cargo test    run tests
cargo fmt     format code
cargo clippy  linting
cargo doc     generate documentation
```

Integrated toolchain for build, test, and docs

12. COMMON GOTCHAS

Warning

Follow language-specific naming conventions
Understand ownership/borrowing/GC model for this language
Use idiomatic patterns don't port code from other languages
Check community guidelines for error handling approach
Profile before optimizing correctness first